

TRƯỜNG ĐẠI HỌC TRÀ VINH
TRƯỜNG KỸ THUẬT VÀ CÔNG NGHỆ
KHOA CÔNG NGHỆ THÔNG TIN



ĐỒ ÁN KẾT THÚC HỌC PHẦN
CÔNG NGHỆ PHẦN MỀM
(MSHP: 220055)

TÊN ĐỀ TÀI
XÂY DỰNG ỨNG DỤNG WEB QUẢN LÝ CHI TIÊU
CÁ NHÂN VỚI JAVA SPRING BOOT

Sinh viên thực hiện:

110122068	Võ Chí Hải	DA22TTD
110122105	Nguyễn Đỗ Thành Lộc	DA22TTD
110122099	Hoàng Tuấn Kiệt	DA22TTD

Giáo viên hướng dẫn: TS. Nguyễn Bảo Ân

Trà Vinh, tháng 6 năm 2025

TRƯỜNG ĐẠI HỌC TRÀ VINH
TRƯỜNG KỸ THUẬT VÀ CÔNG NGHỆ
KHOA CÔNG NGHỆ THÔNG TIN



ĐỒ ÁN KẾT THÚC HỌC PHẦN
CÔNG NGHỆ PHẦN MỀM
(MSHP: 220055)

TÊN ĐỀ TÀI
XÂY DỰNG ỨNG DỤNG WEB QUẢN LÝ CHI TIÊU
CÁ NHÂN VỚI JAVA SPRING BOOT

Sinh viên thực hiện:

110122068	Võ Chí Hải	DA22TTD
110122105	Nguyễn Đỗ Thành Lộc	DA22TTD
110122099	Hoàng Tuấn Kiệt	DA22TTD

Giáo viên hướng dẫn: TS. Nguyễn Bảo Ân

Trà Vinh, tháng 6 năm 2025

[illegible]

(Ký và ghi rõ họ tên)

This image shows a full page of white paper with horizontal ruling lines. The lines are evenly spaced and extend across the width of the page. There are no margins, text, or other markings on the paper.

(Ký và ghi rõ họ tên)

LỜI CẢM ƠN

Để hoàn thành bài báo cáo này, nhóm em xin gửi lời cảm ơn chân thành đến Quý Thầy, Cô thuộc Khoa Công nghệ thông tin, Trường Kỹ thuật và Công nghệ – Trường Đại học Trà Vinh đã tạo điều kiện thuận lợi để chúng em được học tập, rèn luyện và tích lũy kiến thức, kỹ năng nhằm thực hiện bài báo cáo này.

Đặc biệt, em xin gửi lời cảm ơn đến thầy Nguyễn Bảo Ân đã tận tình chỉ dẫn, theo dõi và đưa ra những lời khuyên bổ ích giúp em giải quyết được các vấn đề gặp phải trong quá trình nghiên cứu và hoàn thành đề tài một cách tốt nhất.

Do kiến thức của bản thân còn hạn chế và thiếu kinh nghiệm thực tiễn nên nội dung bài báo cáo khó tránh những thiếu sót. Nhóm chúng em rất mong nhận được sự góp ý, chỉ dạy thêm từ Quý Thầy cô.

Cuối cùng, em xin chúc Quý Thầy Cô luôn thật nhiều sức khỏe và đạt được nhiều thành công trong công việc.

MỤC LỤC

CHƯƠNG 1. GIỚI THIỆU	1
1.1. Giới thiệu đề tài	1
1.2. Mục tiêu của ứng dụng	1
1.3. Lý do chọn đề tài	1
CHƯƠNG 2. PHÂN TÍCH YÊU CẦU	3
2.1. Các chức năng chính của hệ thống	3
2.1.1. Quản lý người dùng	3
2.1.2. Ghi chép thu và chi	3
2.1.3. Xem thống kê và báo cáo tài chính	3
2.2. Các yêu cầu phi chức năng	4
2.2.1. Hiệu năng	4
2.2.2. Bảo mật	4
2.2.3. Tính mở rộng	4
2.2.4. Khả năng bảo trì	5
2.2.5. Tính thân thiện với người dùng	5
CHƯƠNG 3. THIẾT KẾ HỆ THỐNG	6
3.1. Kiến trúc tổng thể hệ thống	6
3.2. Sơ Đồ Kiến Trúc Hệ Thống	7
Hình 1 Sơ đồ kiến trúc của hệ thống	7
3.3. Sơ đồ quan hệ dữ liệu	8
Hình 2 Sơ đồ quan hệ dữ liệu	8
3.4. Thiết kế API swagger mô tả các endpoint cấu trúc request response	9
Bảng 1 API user	9
Bảng 2 API expense	10
Bảng 3 Api incom	11
3.5. Thiết kế giao diện (UI/UX)	12
3.5.1. Giao diện trang đăng nhập	12
3.5.2. Giao diện trang đăng ký	13
3.5.3. Giao diện trang chủ	14
3.5.4. Giao diện menu	15
3.5.5. Giao diện trang quản lý chi tiêu	16
3.5.6. Giao diện trang quản lý thu nhập	17
3.5.7. Trang thống kê chi tiêu	18
CHƯƠNG 4. TRIỂN KHAI CÁC CÔNG CỤ SỬ DỤNG	19
4.1. Danh sách các công cụ sử dụng trong dự án	19
4.1.1. Spring Boot	19
4.1.2. Tailwind CSS	19
4.1.3. MySQL	19
4.1.4. Postman	19
4.1.5. Docker	19
4.1.6. Swagger	20
4.1.7. GitHub	20
4.1.8. JIRA	20
4.1.9. GitHub Actions	20
4.2. Quy trình CI/CD với GitHub Actions	21
4.2.1. CI (Continuous Integration):	21
4.2.2. CD (Continuous Deployment):	21
4.3. Cấu hình Docker và quy trình triển khai ứng dụng	21
4.3.1. Cấu hình Docker:	21
4.3.2. Quy trình triển khai ứng dụng:	21

CHƯƠNG 5. QUẢN LÝ DỰ ÁN	22
5.1. Kế hoạch và tiến độ	22
5.1.1. Thiết lập dự án trên Jira	22
5.1.2. Lập kế hoạch Sprint	22
5.1.3. Theo dõi tiến độ	22
5.2. Phân công nhiệm vụ của từng thành viên trong nhóm	22
Bảng 4 Bảng phân công công việc	22
CHƯƠNG 6. KIỂM THỬ	23
6.1. Chiến lược kiểm thử	23
6.1.1 Mục tiêu	23
6.1.2 Phạm vi kiểm thử	23
6.1.3 Loại kiểm thử áp dụng	23
6.2. Công cụ sử dụng và kết quả kiểm thử API	24
6.2.1 kết quả kiểm thử API	25
CHƯƠNG 7. ĐÁNH GIÁ VÀ KẾT LUẬN	39
7.1. Những khó khăn gặp phải trong quá trình thực hiện	39
7.2. Bài học rút ra và đề xuất cải thiện trong tương lai	39
CHƯƠNG 8. PHỤ LỤC	41
8.1. Hướng dẫn cài đặt và chạy ứng dụng	41
8.1.1. Cài đặt môi trường	41
Cài đặt JDK	41
Truy cập:	41
8.1.2. Cài đặt XAMPP	41
8.1.5. Clone Ứng Dụng & Truy Cập	42
8.1.6. Cài Đặt Thư Viện Phụ Thuộc	42
8.1.7. Khởi Động Ứng Dụng	42
8.1.8. Ứng dụng sẽ chạy tại địa chỉ:	42
8.2. Link GitHub và repository	42

Danh mục hình ảnh và bảng biểu

Hình 1 Sơ đồ kiến trúc của hệ thống	7
Hình 2 Sơ đồ quan hệ dữ liệu	8
Bảng 1 API user	9
Bảng 2 API expense	10
Bảng 3 Api incom	11
Hình 3 Trang đăng nhập	12
Hình 4 Trang đăng ký	13
Hình 5 Trang chủ	14
Hình 6 Menu	15
Hình 7 Trang quản lý chi tiêu	16
Hình 8 Trang quản lý thu nhập	17
Hình 9 Trang thống kê chi tiêu	18
Bảng 4 Bảng phân công công việc	22
Bảng 5 Công cụ kiểm thử API	24
Hình 10 Các endpoint trong dự án	24
Hình 11 API hiển thị tất cả user	25
Hình 12 API Hiển thị user theo id	26
Hình 13 API đăng ký tài khoản	27
Hình 14 API tạo mới thu nhập	28
Hình 15 API hiển thị tất cả thu nhập	29
Hình 16 API hiển thị thu nhập theo id	30
Hình 17 API xóa thu nhập	31
Hình 19 API tạo mới chi tiêu	33
Hình 20 API hiển thị tất cả chi tiêu	34
Hình 21 API hiển thị chi tiêu theo id	35
Hình 22 API xóa chi tiêu	36
Hình 23 API chi tiêu không tìm thấy	37
Hình 24 API tổng chi tiêu từng tháng	38

CHƯƠNG 1. GIỚI THIỆU

1.1. Giới thiệu đề tài

Tên đề tài: Xây dựng ứng dụng web quản lý chi tiêu cá nhân sử dụng Spring Boot

Chủ đề: Phát triển ứng dụng hỗ trợ người dùng ghi chép, phân loại và phân tích các khoản thu – chi cá nhân thông qua giao diện web trực quan, thân thiện, áp dụng công nghệ Spring Boot.

1.2. Mục tiêu của ứng dụng

Ứng dụng hướng đến việc hỗ trợ người dùng:

Quản lý thu nhập và chi tiêu hàng ngày.

Hiện thị báo cáo thống kê theo tuần/tháng/năm bằng biểu đồ.

Đặt giới hạn chi tiêu theo tháng và cảnh báo khi vượt hạn mức.

1.3. Lý do chọn đề tài

Trong đời sống hiện đại, việc quản lý tài chính cá nhân ngày càng trở thành một kỹ năng quan trọng, đặc biệt đối với sinh viên, người đi làm và những người trẻ tuổi mới bắt đầu xây dựng cuộc sống độc lập. Với sự phát triển của công nghệ, nhu cầu số hóa trong quản lý chi tiêu không chỉ là một xu hướng mà còn là một giải pháp thiết yếu giúp người dùng kiểm soát tốt hơn thói quen tiêu dùng, từ đó hướng tới một lối sống tiết kiệm và ổn định về tài chính.

Tuy nhiên, trên thực tế, không phải ai cũng có thể duy trì việc ghi chép thu và chi một cách đều đặn bằng các phương pháp thủ công như sổ tay hay bảng tính Excel. Những cách này tuy đơn giản nhưng thiếu tính linh hoạt, khó đồng bộ và dễ gây nhầm lẫn. Mặt khác, nhiều ứng dụng quản lý chi tiêu hiện có trên thị trường thường tích hợp quá nhiều tính năng phức tạp, yêu cầu người dùng phải làm quen trong thời gian dài hoặc gặp rào cản ngôn ngữ (đa phần là tiếng Anh). Điều đó khiến người dùng phổ thông dễ nản và từ bỏ việc theo dõi tài chính cá nhân.

Chính vì vậy, nhóm đã quyết định lựa chọn đề tài “Ứng dụng quản lý chi tiêu cá nhân” với mục tiêu xây dựng một hệ thống đơn giản, dễ sử dụng, giao diện thân thiện, tập trung vào những chức năng thiết yếu nhất như: ghi chép thu chi, phân loại khoản mục, xem báo cáo thống kê và cảnh báo khi vượt ngân sách. Ứng dụng được xây dựng dưới dạng website để người dùng có thể dễ dàng truy cập từ mọi thiết bị có trình duyệt, đồng thời sử dụng Spring Boot – một framework mạnh mẽ, hiện đại và phù hợp với mô hình phát triển web chuyên nghiệp, giúp đảm bảo hiệu suất của hệ thống.

Bên cạnh đó, việc lựa chọn đề tài này còn xuất phát từ mong muốn củng cố và nâng cao kỹ năng lập trình web backend bằng Java – một trong những ngôn ngữ phổ biến nhất hiện nay trong lĩnh vực phát triển phần mềm doanh nghiệp. Việc áp dụng Spring Boot không chỉ giúp nhóm hiểu rõ hơn về cấu trúc và luồng xử lý của một ứng dụng web, mà còn rèn luyện được nhiều kỹ năng quan trọng như thiết kế API RESTful, quản lý cơ sở dữ liệu quan hệ, triển khai ứng dụng thực tế, cũng như xử lý các vấn đề liên quan đến bảo mật và xác thực người dùng.

Ngoài ra, đề tài này còn mang tính thực tiễn cao vì giải quyết được một nhu cầu phổ biến trong cuộc sống hàng ngày của nhiều người: quản lý chi tiêu cá nhân một cách hiệu quả. Thay vì sử dụng sổ tay hoặc ghi chú rời rạc, người dùng có thể lưu trữ thông tin tài chính của mình một cách khoa học, có hệ thống và dễ dàng truy xuất khi cần. Từ đó, họ sẽ kiểm soát tốt hơn dòng tiền của bản thân, đưa ra các quyết định tài chính hợp lý và hướng tới việc tiết kiệm hoặc đầu tư hiệu quả hơn.

Về mặt kỹ thuật, hệ thống sử dụng cơ sở dữ liệu quan hệ như MySQL để lưu trữ thông tin người dùng, các khoản thu chi và phân loại mục tiêu ngân sách. Các chức năng được xây dựng dưới dạng API RESTful giúp đảm bảo khả năng mở rộng và tích hợp dễ dàng trong tương lai. Giao diện người dùng (Frontend) được thiết kế theo hướng đơn giản, trực quan, dễ thao tác kể cả với những người không am hiểu công nghệ. Đặc biệt, hệ thống cũng tích hợp tính năng cảnh báo khi người dùng có xu hướng chi tiêu vượt quá mức thu nhập đã đặt ra, góp phần nâng cao ý thức quản lý chi tiêu cá nhân.

CHƯƠNG 2. PHÂN TÍCH YÊU CẦU

2.1. Các chức năng chính của hệ thống

2.1.1. Quản lý người dùng

Đăng ký tài khoản: người dùng tạo tài khoản mới với các thông tin bao gồm: tên người dùng và mật khẩu.

Đăng nhập/đăng xuất: xác thực người dùng vào hệ thống.

Bảo mật thông tin người dùng bằng cách mã hóa mật khẩu (dùng BCrypt).

2.1.2. Ghi chép thu và chi

Giao diện đơn giản để nhập thông tin thu nhập hoặc khoản chi tiêu.

Các trường thông tin bao gồm: tên giao dịch (thu/chi), số tiền, ngày thực hiện.

Cho phép chỉnh sửa hoặc xóa giao dịch đã tạo.

2.1.3. Xem thống kê và báo cáo tài chính

Thống kê tổng thu – chi theo tuần, tháng, năm.

Hiển thị số dư hiện tại của người dùng.

Biểu đồ hình tròn và cột thể hiện tỷ lệ các khoản chi theo danh mục.

Biểu đồ đường hiển thị biến động số dư theo thời gian.

2.2. Các yêu cầu phi chức năng

Bên cạnh các chức năng chính, hệ thống cũng cần đáp ứng các yêu cầu phi chức năng nhằm đảm bảo trải nghiệm người dùng và chất lượng phần mềm.

2.2.1. Hiệu năng

Hệ thống phải phản hồi nhanh (dưới 2 giây) cho các thao tác như ghi giao dịch, tìm kiếm, thống kê.

Cơ sở dữ liệu được tối ưu bằng chỉ mục (index) cho truy vấn nhanh.

2.2.2. Bảo mật

Mã hóa mật khẩu người dùng khi lưu vào cơ sở dữ liệu.

Sử dụng Spring Security để bảo vệ tài nguyên backend.

Kiểm tra dữ liệu đầu vào để tránh tấn công SQL Injection, XSS.

2.2.3. Tính mở rộng

Hệ thống được thiết kế theo mô hình MVC (Model – View – Controller) kết hợp với kiến trúc RESTful API, trong đó backend sử dụng framework Spring Boot. Mô hình này giúp tổ chức mã nguồn một cách rõ ràng và tách biệt các thành phần của ứng dụng: phần xử lý logic nghiệp vụ (Model), phần giao tiếp và điều hướng luồng dữ liệu (Controller), và phần giao diện người dùng (View). Bằng cách tách rời các lớp chức năng, hệ thống trở nên dễ bảo trì, mở rộng, và thuận tiện trong việc kiểm thử từng phần riêng biệt.

Sự kết hợp giữa mô hình MVC và kiến trúc RESTful không chỉ giúp tăng cường tính linh hoạt mà còn hỗ trợ quá trình phát triển phần mềm theo hướng hiện đại, dễ tích hợp với các công nghệ mới. Việc sử dụng Spring Boot giúp giảm bớt cấu hình phức tạp, đẩy nhanh tiến độ phát triển và triển khai ứng dụng. Nhờ đó, hệ thống có thể đáp ứng tốt các yêu cầu kỹ thuật cũng như nâng cao trải nghiệm người dùng.

2.2.4. Khả năng bảo trì

Viết mã sạch, dễ hiểu, có tài liệu chú thích rõ ràng.

Cấu trúc thư mục rõ ràng theo chuẩn Spring Boot: controller, service, repository, model, config.

Dễ nâng cấp, kiểm thử và chỉnh sửa khi cần thiết.

2.2.5. Tính thân thiện với người dùng

Giao diện người dùng của hệ thống được thiết kế theo hướng tối giản, trực quan và thân thiện với người sử dụng, đặc biệt phù hợp với những người không am hiểu nhiều về công nghệ. Các thành phần được bố trí hợp lý, giúp người dùng dễ dàng thao tác mà không cần mất nhiều thời gian tìm hiểu. Các nút chức năng được đặt ở vị trí dễ nhìn, dễ truy cập và sử dụng các nhãn rõ ràng, giúp người dùng hiểu ngay chức năng của từng phần mà không cần hướng dẫn phức tạp.

Hệ thống áp dụng thiết kế responsive, đảm bảo khả năng hiển thị tốt trên nhiều thiết bị khác nhau như máy tính, máy tính bảng và điện thoại di động. Giao diện tự động điều chỉnh bố cục, kích thước chữ và hình ảnh để phù hợp với độ phân giải của từng thiết bị, từ đó mang lại trải nghiệm đồng nhất cho người dùng ở mọi môi trường. Việc này không chỉ tăng tính tiện dụng mà còn nâng cao khả năng tiếp cận của hệ thống đối với đa dạng người dùng.

Bên cạnh đó, màu sắc được sử dụng một cách hợp lý, hài hòa và có chủ đích nhằm phân biệt các thành phần chức năng, đồng thời tạo cảm giác dễ chịu khi sử dụng lâu dài. Các biểu tượng (icon) quen thuộc được tích hợp để minh họa các chức năng như thêm, sửa, xóa, giúp tăng tính trực quan. Ngoài ra, hệ thống còn sử dụng các biểu đồ và hình ảnh trực quan để trình bày thông tin chi tiêu một cách sinh động, giúp người dùng dễ dàng nắm bắt và phân tích dữ liệu tài chính của mình mà không cần hiểu sâu về số liệu.

CHƯƠNG 3. THIẾT KẾ HỆ THỐNG

3.1. Kiến trúc tổng thể hệ thống

Hệ thống được xây dựng theo kiến trúc Monolithic, trong đó toàn bộ giao diện người dùng, xử lý nghiệp vụ và truy cập dữ liệu được tích hợp trong cùng một ứng dụng duy nhất. Kiến trúc này đơn giản trong triển khai và vận hành, phù hợp với các hệ thống có quy mô vừa và nhỏ, giúp tối ưu hiệu suất khi triển khai ban đầu và dễ dàng kiểm soát logic hệ thống trong một khối thống nhất.

Các thành phần chính bao gồm:

Frontend:

Giao diện người dùng được xây dựng bằng Thymeleaf – một template engine chạy phía server, cho phép render HTML động dựa trên dữ liệu trả về từ backend. Với Thymeleaf, các trang HTML được sinh ra trực tiếp từ server, giúp đồng bộ giữa giao diện và logic xử lý, đồng thời giảm độ phức tạp khi phát triển giao diện.

Để thiết kế giao diện, hệ thống sử dụng Tailwind CSS, một framework CSS hiện đại theo hướng utility-first. Tailwind hỗ trợ lập trình viên dễ dàng tạo nên các giao diện thân thiện, nhất quán, phù hợp với nhiều độ phân giải khác nhau mà không cần viết CSS thủ công quá nhiều. Giao diện hệ thống đạt tính responsive, cho phép hoạt động tốt trên cả máy tính và thiết bị di động.

Backend:

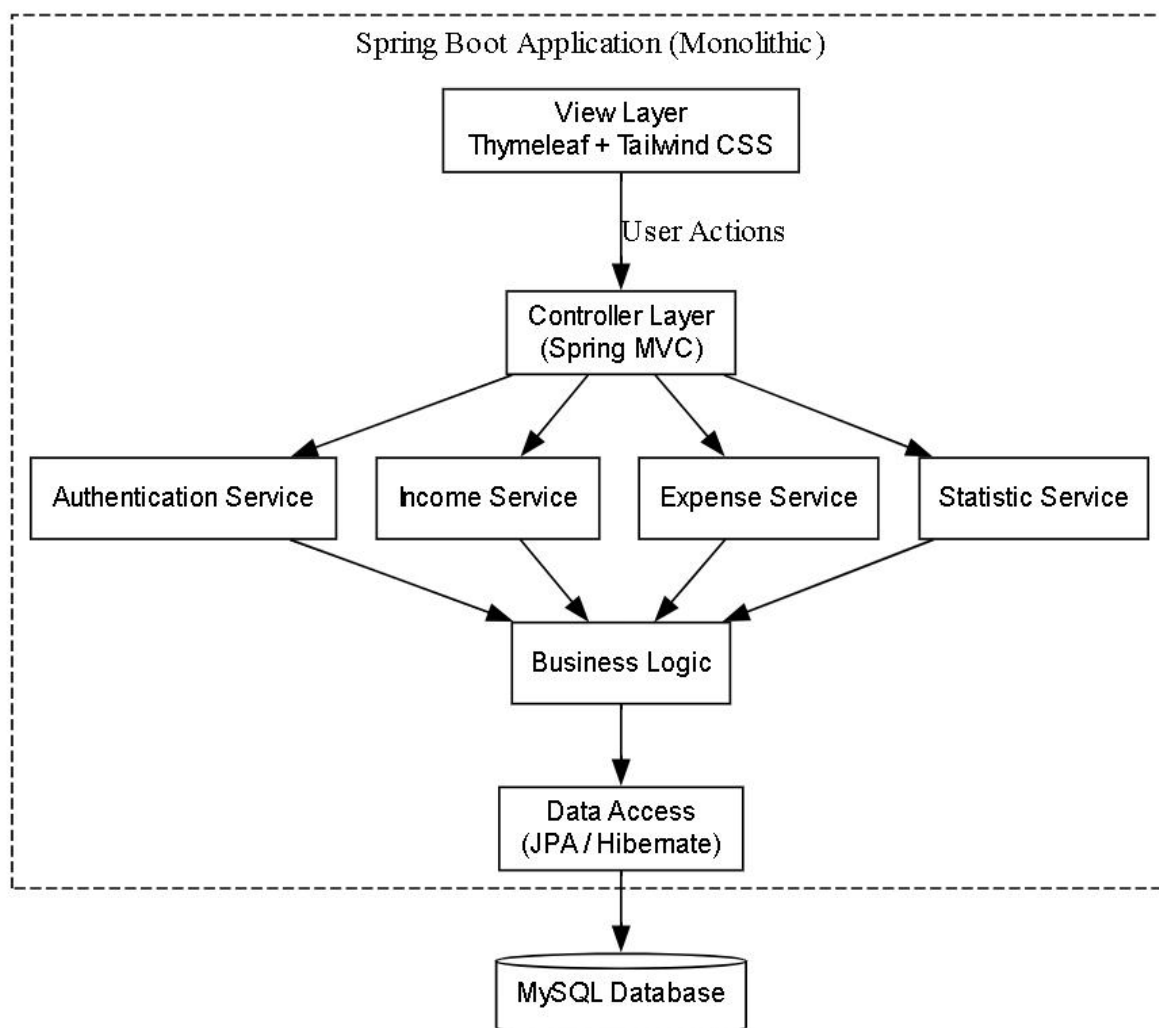
Phần backend được xây dựng với Spring Boot, một framework mạnh mẽ thuộc hệ sinh thái Java. Backend chịu trách nhiệm xử lý logic nghiệp vụ, thực hiện các chức năng như xác thực người dùng, quản lý phiên làm việc, tiếp nhận các yêu cầu từ giao diện người dùng và xử lý dữ liệu tương ứng.

Toàn bộ quy trình xử lý dữ liệu, tính toán, phân tích và phản hồi kết quả đều diễn ra ở backend. Backend và frontend được tích hợp chặt chẽ trong cùng một ứng dụng Monolithic, giúp giảm độ trễ và tăng độ đồng bộ giữa hai phần này.

Database:

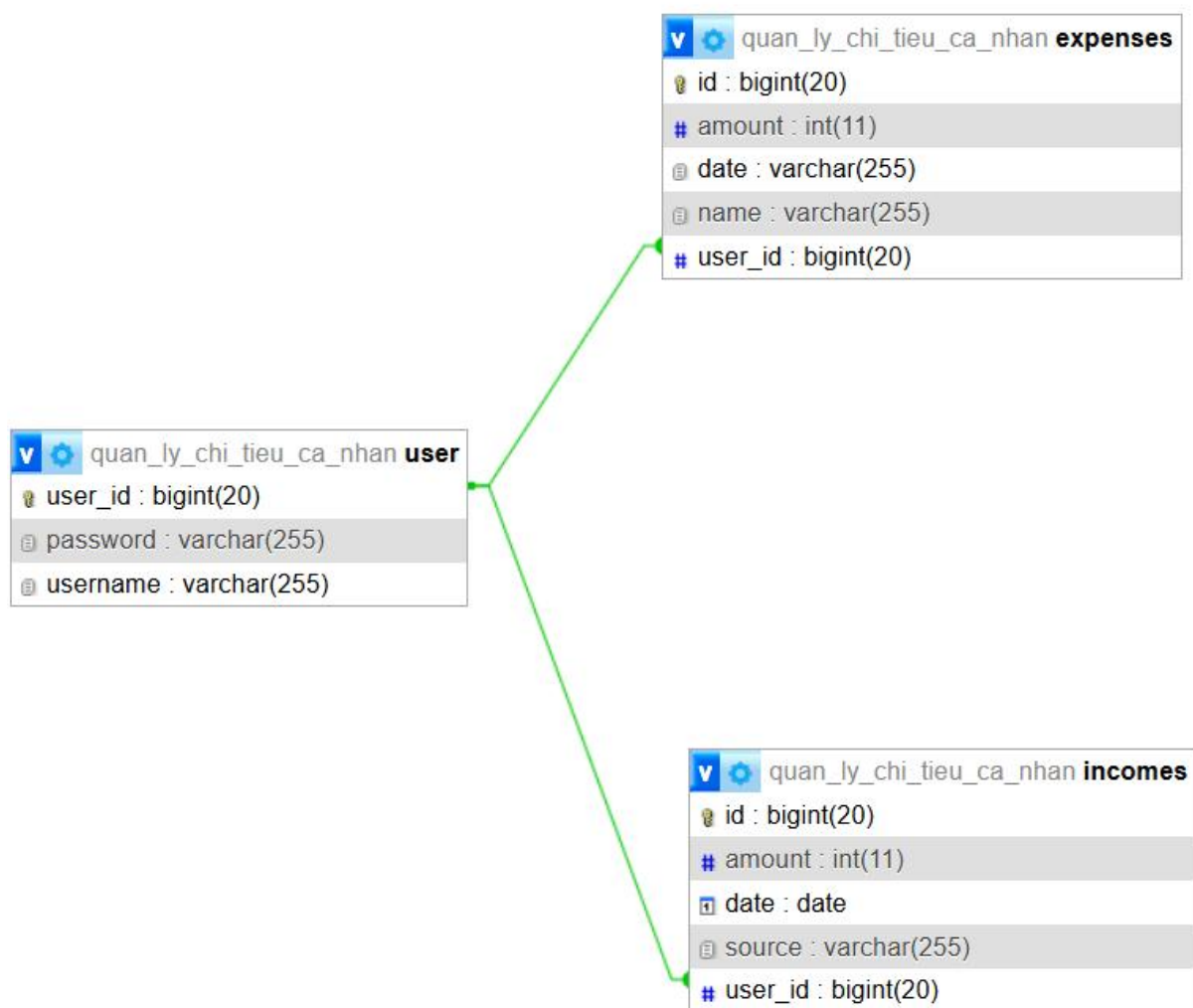
Hệ thống sử dụng MySQL làm hệ quản trị cơ sở dữ liệu chính. MySQL lưu trữ toàn bộ thông tin người dùng, dữ liệu thu chi cá nhân, danh mục các loại chi tiêu, cũng như các thiết lập cấu hình hệ thống.

3.2. Sơ Đồ Kiến Trúc Hệ Thống



Hình 1 Sơ đồ kiến trúc của hệ thống

3.3. Sơ đồ quan hệ dữ liệu



Hình 2 Sơ đồ quan hệ dữ liệu

3.4. Thiết kế API swagger mô tả các endpoint cấu trúc request response

Endpoint	Phương thức	Mô tả	Yêu cầu (Request)	Phản hồi (Response)
/api/users/register	POST	Đăng ký người dùng bằng username và password	{ “username”: “user1”, “password”: “123”}	- 200 OK: Đăng ký thành công. -
/api/users	GET	Hiển thị tất cả người dùng	- Không có request body.	- 200 OK: Danh sách tất cả người dùng
/api/users/{username}	GET	Hiển thị người dùng theo username	-Ko có request body	- 200 OK: Người dùng theo username -404 NOT FOUND: Ko tìm thấy

Bảng 1 API user

Endpoint	Phương thức	Mô tả	.Yêu cầu (Request)	Phản hồi (Response)
/api/expenses	GET	Xem tất cả các khoản chi tiêu của người dùng	Không có request body.	200 OK: Thành công
/api/expenses/{id}	GET	Xem các khoản chi tiêu theo id	Không có request body.	- 200 OK: Thành công - 404 NOT FOUND: Ko tìm thấy
/api/expenses/monthly	GET	Xem tổng các khoản chi tiêu theo từng tháng	Không có request body.	- 200 OK: Thành công
/api/expenses	POST	Tạo mới một chi tiêu	{ "expense": "tiền điện", "amount": "250000", "date": "2025-06-10" }	- 200 OK: Thành công
/api/expenses	DELETE	Xóa một khoản chi tiêu	Không có request body	- 200 OK: Thành công

Bảng 2 API expense

Endpoint	Phương thức	Mô tả	.Yêu cầu (Request)	Phản hồi (Response)
/api/incomes	GET	Xem tất cả các khoản thu nhập	Không có request body.	200 OK: Thành công
/api/incomes/{id}	GET	Xem các khoản thu nhập theo id	Không có request body.	- 200 OK: Thành công - 404 NOT FOUND: Ko tìm thấy
/api/incomes	POST	Tạo mới một thu nhập	{ "source": "tiền lương", "amount": "7000000", "date": "2025-06-30" }	- 200 OK: Thành công
/api/incomes	DELETE	Xóa một khoản thu nhập	Không có request body	- 200 OK: Thành công - 404 NOT FOUND: Ko tìm thấy

Bảng 3 Api incom

3.5. Thiết kế giao diện (UI/UX)

3.5.1. Giao diện trang đăng nhập

The image shows a login form with a light gray background. At the top, the title "Đăng Nhập" is centered in bold black text. Below it, the label "Tài khoản:" is followed by a light gray rectangular input field. Then, the label "Mật khẩu:" is followed by another light gray rectangular input field. Below these fields is a prominent blue rectangular button with the text "Đăng Nhập" in white. At the bottom, the text "Đăng ký" is displayed in a smaller, blue font.

Hình 3 Trang đăng nhập

Giao diện đăng nhập của hệ thống bao gồm: các trường nhập liệu cho tài khoản và mật khẩu, cùng với nút “Đăng Nhập” để xác thực thông tin người dùng.

Ngoài ra, người dùng chưa có tài khoản có thể nhấn vào liên kết “Đăng ký” để chuyển đến trang tạo tài khoản mới.

Thiết kế đơn giản, dễ sử dụng, đảm bảo trải nghiệm người dùng thuận tiện và hiệu quả.

3.5.2. Giao diện trang đăng ký



The image shows a registration form titled "Đăng Ký". It contains two input fields: "Tài khoản:" (Account) and "Mật khẩu:" (Password). Below these fields is a blue button labeled "Đăng Ký". At the bottom, there is a link that says "Đã có tài khoản" (Already have an account).

Hình 4 Trang đăng ký

Giao diện đăng ký người dùng mới của hệ thống bao gồm các trường nhập liệu cơ bản như “Tài khoản” và “Mật khẩu”, cho phép người dùng tạo tài khoản để truy cập vào hệ thống.

Nút “Đăng Ký” được thiết kế nổi bật nhằm thu hút sự chú ý và thúc đẩy hành động. Ngoài ra, liên kết “Đã có tài khoản” giúp người dùng dễ dàng chuyển sang trang đăng nhập nếu họ đã đăng ký trước đó.

Thiết kế đơn giản, trực quan, phù hợp với trải nghiệm người dùng trên cả máy tính và thiết bị di động.

3.5.3. Giao diện trang chủ



Hình 5 Trang chủ

Giao diện chính của hệ thống quản lý chi tiêu sau khi người dùng đăng nhập thành công.

Trang chủ đóng vai trò là điểm bắt đầu cho mọi thao tác quản lý tài chính cá nhân, cung cấp các chức năng như xem báo cáo chi tiêu, thêm giao dịch mới, và truy cập các mục quản lý khác.

Thông điệp “Chào mừng đến với Quản lý chi tiêu” thể hiện sự thân thiện và định hướng người dùng vào trải nghiệm chính của ứng dụng.

3.5.4. Giao diện menu



Hình 6 Menu

Giao diện menu chính của hệ thống quản lý tài chính cá nhân, được thiết kế với bố cục hai phần: bên trái là danh sách các chức năng chính, bên phải là phần hiển thị nội dung tương ứng.

Các mục trong menu bao gồm “Quản lý chỉ tiêu”, “Quản lý thu nhập”, “Thoát” và “Đăng xuất”, cho phép người dùng dễ dàng điều hướng giữa các chức năng của hệ thống quản lý chỉ tiêu.

Thiết kế màu sắc tương phản giữa nền xanh và chữ trắng giúp tăng khả năng nhận diện và cải thiện trải nghiệm người dùng.

3.5.5 . Giao diện trang quản lý chi tiêu

[Quay lại](#)

Quản lý chi tiêu

Tên Chi Tiêu

Số Tiền

dd/mm/yyyy

Thêm Chi Tiêu

Tên Chi Tiêu	Số Tiền	Ngày	Hành Động
tiền học phí	11000000	2025-05-22	Xóa Sửa
tiền trọ	2500000	2025-05-25	Xóa Sửa
tiền đi net	325000	2025-05-28	Xóa Sửa
tiền xăng	400000	2025-05-24	Xóa Sửa
tiền mua trà sữa	250000	2025-05-29	Xóa Sửa

Tổng chi tiêu: 14475000 đ

Hình 7 Trang quản lý chi tiêu

Trang quản lý chi tiêu hiển thị danh sách các khoản chi tiêu cá nhân dưới dạng bảng, bao gồm các cột: “Tên Chi Tiêu”, “Số Tiền”, “Ngày” và “Hành Động”. Người dùng có thể thêm khoản chi mới bằng nút “Thêm Chi Tiêu” và thực hiện các thao tác như “Xóa” hoặc “Sửa” đối với từng mục đã nhập.

Dữ liệu được trình bày rõ ràng, hỗ trợ người dùng theo dõi và kiểm soát chi tiêu hàng ngày một cách hiệu quả. Tổng chi tiêu được tính toán và hiển thị ở cuối bảng, giúp người dùng nắm bắt nhanh tình hình tài chính cá nhân.

3.5.6. Giao diện trang quản lý thu nhập

[Quay lại](#)

Quản lý thu nhập

Tên Thu

Số Tiền

dd/mm/yyyy

Thêm Thu Nhập

Nguồn Thu	Số Tiền	Ngày	Hành Động
tiền lương	5000000	2025-05-22	Xóa Sửa
tiền tăng ca	3000000	2025-05-24	Xóa Sửa
tiền Freelance	6500000	2025-05-26	Xóa Sửa
tiền thưởng	4700000	2025-05-29	Xóa Sửa
tiền giao hàng	8000000	2025-05-30	Xóa Sửa

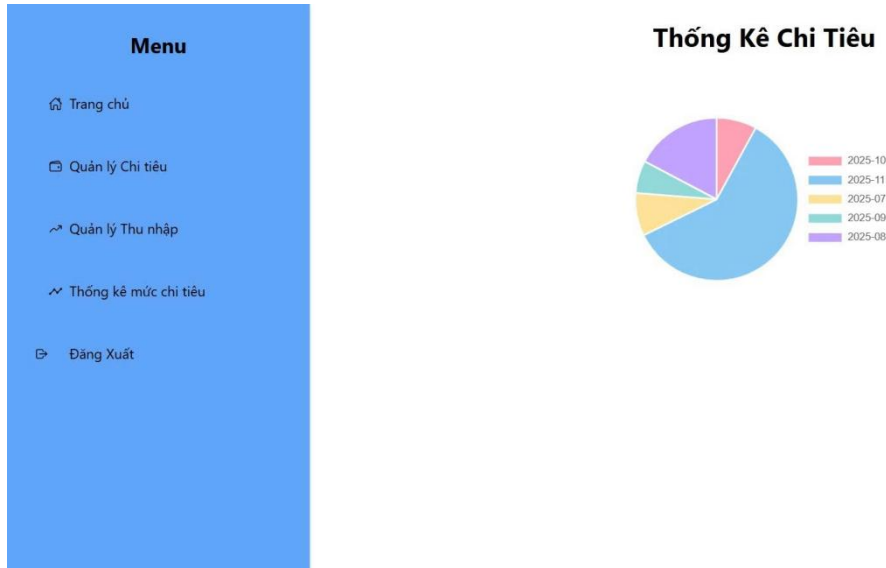
Tổng thu nhập: 14475000 đ

Hình 8 Trang quản lý thu nhập

Trang quản lý thu nhập cho phép người dùng ghi nhận và theo dõi các nguồn thu nhập cá nhân theo từng ngày. Giao diện bao gồm các trường nhập liệu như “Nguồn Thu”, “Số Tiền” và “Ngày”, cùng với bảng hiển thị danh sách các khoản thu đã được thêm vào. Mỗi dòng trong bảng đều có các nút “Xóa” và “Sửa” để người dùng dễ dàng chỉnh sửa hoặc loại bỏ thông tin khi cần thiết.

Tổng thu nhập được tính toán và hiển thị rõ ràng ở cuối bảng, giúp người dùng có cái nhìn tổng quan về tình hình tài chính của mình.

3.5.7. Trang thống kê chi tiêu



Hình 9 Trang thống kê chi tiêu

Trang thống kê chi tiêu là một phần quan trọng trong hệ thống quản lý tài chính cá nhân, giúp người dùng có cái nhìn tổng quan và trực quan về thói quen chi tiêu của mình theo từng tháng. Giao diện được chia thành hai phần chính: thanh menu điều hướng bên trái và biểu đồ hình tròn (pie chart) bên phải.

Thanh menu bên trái bao gồm các mục chức năng như “Trang chủ”, “Quản lý Chi tiêu”, “Quản lý Thu nhập”, “Thống kê mức chi tiêu” và “Đăng Xuất”, cho phép người dùng dễ dàng chuyển đổi giữa các phần của hệ thống.

Biểu đồ hình tròn ở phần chính giữa giao diện thể hiện tỷ lệ chi tiêu theo từng tháng, với mỗi màu sắc đại diện cho một tháng cụ thể (ví dụ: tháng 7, 8, 9, 10, 11 năm 2015). Việc sử dụng biểu đồ giúp người dùng dễ dàng nhận biết tháng nào có mức chi tiêu cao nhất hoặc thấp nhất, từ đó đưa ra các quyết định điều chỉnh phù hợp cho kế hoạch tài chính cá nhân trong tương lai.

Tính năng thống kê này không chỉ hỗ trợ việc theo dõi mà còn đóng vai trò như một công cụ phân tích, giúp người dùng hiểu rõ hơn về hành vi tài chính của bản thân, từ đó xây dựng thói quen chi tiêu hợp lý và tiết kiệm hiệu quả hơn.

CHƯƠNG 4. TRIỂN KHAI CÁC CÔNG CỤ SỬ DỤNG

4.1. Danh sách các công cụ sử dụng trong dự án

4.1.1. Spring Boot

Mô tả: Spring Boot là một framework Java mạnh mẽ, giúp xây dựng các ứng dụng web một cách nhanh chóng và dễ dàng. Dự án sử dụng Spring Boot để xây dựng API backend, xử lý logic nghiệp vụ, và tương tác với cơ sở dữ liệu.

Vai trò trong dự án: Xử lý các yêu cầu từ phía người dùng, quản lý dữ liệu chi tiêu, và cung cấp API cho các tính năng của ứng dụng.

4.1.2. Tailwind CSS

Mô tả: Tailwind CSS là một framework CSS tiện lợi cho phép xây dựng giao diện người dùng (UI) theo cách linh hoạt và tùy chỉnh.

Vai trò trong dự án: Dùng để thiết kế giao diện người dùng cho ứng dụng, tạo các thành phần UI dễ dàng và đẹp mắt mà không cần viết quá nhiều CSS thủ công.

4.1.3. MySQL

Mô tả: MySQL là một hệ quản trị cơ sở dữ liệu quan hệ mã nguồn mở.

Vai trò trong dự án: Lưu trữ dữ liệu chi tiêu cá nhân, thông tin người dùng, và các thông tin liên quan đến các giao dịch tài chính.

4.1.4. Postman

Mô tả: Postman là một công cụ phát triển API phổ biến, dùng để kiểm thử các API và gửi yêu cầu HTTP.

Vai trò trong dự án: Sử dụng để kiểm thử API backend của ứng dụng, đảm bảo rằng các yêu cầu được thực thi chính xác và trả về kết quả đúng đắn

4.1.5. Docker

Mô tả: Docker là một nền tảng để phát triển, vận hành và phân phối các ứng dụng trong các container.

Vai trò trong dự án: Sử dụng Docker để đóng gói ứng dụng và các môi trường phát triển thành các container độc lập, giúp triển khai ứng dụng dễ dàng và đồng nhất trên các môi trường khác nhau.

4.1.6. Swagger

Mô tả: Swagger là một bộ công cụ mã nguồn mở được sử dụng rộng rãi để thiết kế, xây dựng, ghi tài liệu và tiêu chuẩn hóa các API RESTful. Với Swagger, các nhà phát triển có thể dễ dàng tạo ra tài liệu API một cách tự động từ mã nguồn backend, giúp tiết kiệm thời gian và giảm thiểu sai sót trong quá trình viết tài liệu thủ công.

Vai trò trong dự án: Tạo và duy trì tài liệu API cho backend, giúp các nhà phát triển khác hiểu rõ hơn về các endpoints và cách sử dụng chúng.

4.1.7. GitHub

Mô tả: GitHub là một nền tảng lưu trữ mã nguồn dựa trên Git, cung cấp các công cụ mạnh mẽ để quản lý phiên bản, cộng tác nhóm và kiểm soát chất lượng phần mềm. Ngoài việc lưu trữ mã nguồn, GitHub còn hỗ trợ các tính năng như pull request, issue tracking, và tích hợp CI/CD.

Vai trò trong dự án: Quản lý mã nguồn của dự án, đồng bộ hóa công việc giữa các thành viên trong nhóm và theo dõi lịch sử thay đổi của mã nguồn.

4.1.8. JIRA

Mô tả: JIRA là một phần mềm quản lý dự án chuyên nghiệp do Atlassian phát triển, được sử dụng phổ biến trong các nhóm phát triển phần mềm theo phương pháp Agile. JIRA cho phép tạo và quản lý các task, bug, story, và epic, đồng thời cung cấp các bảng Kanban hoặc Scrum để theo dõi tiến độ công việc.

Vai trò trong dự án: Quản lý và theo dõi tiến độ phát triển, các lỗi phần mềm, và các công việc trong suốt vòng đời của dự án.

4.1.9. GitHub Actions

Mô tả: GitHub Actions là một tính năng tích hợp sẵn trong GitHub, cho phép người dùng thiết lập các workflow tự động để thực hiện các tác vụ như build, test, và deploy ứng dụng. Đây là một công cụ CI/CD mạnh mẽ, giúp giảm thiểu công việc thủ công và tăng tốc độ phát triển phần mềm.

Vai trò trong dự án: Cấu hình quy trình CI/CD để tự động hóa việc build, kiểm thử và triển khai ứng dụng mỗi khi có thay đổi mã nguồn.

4.2. Quy trình CI/CD với GitHub Actions

Quy trình CI/CD của dự án sử dụng GitHub Actions nhằm tự động hóa quá trình xây dựng, kiểm thử và triển khai ứng dụng:

4.2.1. CI (Continuous Integration):

Khi có thay đổi mã nguồn được đẩy lên repository trên GitHub, GitHub Actions sẽ tự động thực hiện các bước sau:

Xây dựng lại ứng dụng.

Kiểm thử mã nguồn (unit test, integration test).

Kiểm tra mã nguồn với các công cụ như SonarQube.

4.2.2. CD (Continuous Deployment):

Sau khi quá trình kiểm thử thành công, GitHub Actions sẽ tự động triển khai ứng dụng vào môi trường sản xuất hoặc staging. Quá trình này được thực hiện thông qua việc chạy các script Docker để xây dựng và triển khai ứng dụng.

4.3. Cấu hình Docker và quy trình triển khai ứng dụng

4.3.1. Cấu hình Docker:

Dockerfile: Một tệp Dockerfile đã được sử dụng để xây dựng container cho ứng dụng Spring Boot. Tệp này chứa các lệnh để tạo môi trường phù hợp, cài đặt các phụ thuộc và chạy ứng dụng trong container.

docker-compose.yml: Docker Compose được sử dụng để định nghĩa và chạy các ứng dụng Docker trong môi trường phát triển, bao gồm cả ứng dụng backend (Spring Boot) và cơ sở dữ liệu MySQL.

4.3.2. Quy trình triển khai ứng dụng:

Bước 1: Ứng dụng được đóng gói vào một Docker container.

Bước 2: Docker container được triển khai lên một môi trường staging hoặc production (có thể sử dụng các dịch vụ như AWS, DigitalOcean hoặc máy chủ riêng).

Bước 3: Sau khi triển khai, người dùng có thể truy cập ứng dụng web qua trình duyệt để thực hiện các chức năng quản lý chi tiêu cá nhân.

CHƯƠNG 5. QUẢN LÝ DỰ ÁN

5.1. Kế hoạch và tiến độ

5.1.1. Thiết lập dự án trên Jira

Tạo dự án mới với loại "Scrum software project".

Xác định backlog gồm các chức năng/feature cần triển khai.

Tạo các issue tương ứng: Story, Task, Bug, Sub-task.

5.1.2. Lập kế hoạch Sprint

Chọn các issue từ backlog để đưa vào sprint.

Ước lượng độ khó bằng Story Points.

Thiết lập thời gian Sprint (1 tuần, 2 tuần, 3 tuần).

5.1.3. Theo dõi tiến độ

Sử dụng Sprint Board để theo dõi trạng thái từng công việc: To Do,

In Progress, Done.

Kiểm tra biểu đồ Burndown Chart để đánh giá tiến độ thực tế so với kế hoạch.

5.2. Phân công nhiệm vụ của từng thành viên trong nhóm

Họ và tên	Vai trò	Nhiệm vụ
Võ Chí Hải	Scrum Master, Developer, DevOps	Quản lý Sprint, theo dõi tiến độ trên Jira Phát triển Backend và Frontend. Triển khai ứng dụng (Deploy lên Docker)
Nguyễn Đỗ Thành Lộc	Designer	Thiết kế UI/UX với Figma
Hoàng Tuấn Kiệt	Tester	Viết test case, kiểm thử chức năng

Bảng 4 Bảng phân công công việc

CHƯƠNG 6. KIỂM THỬ

6.1. Chiến lược kiểm thử

Chiến lược kiểm thử là kế hoạch tổng thể xác định cách thức tiến hành kiểm thử phần mềm nhằm đảm bảo chất lượng sản phẩm. Đối với ứng dụng web quản lý chi tiêu cá nhân, chiến lược kiểm thử được thiết kế như sau:

6.1.1 Mục tiêu

Đảm bảo các API backend hoạt động đúng logic nghiệp vụ (quản lý tài khoản, ghi nhận chi tiêu, xem báo cáo, v.v).

Phát hiện và xử lý lỗi sớm trong quá trình phát triển.

Tự động hóa kiểm thử để rút ngắn thời gian kiểm thử thủ công.

6.1.2 Phạm vi kiểm thử

Chỉ tập trung vào kiểm thử **API backend** (RESTful API).

Không kiểm thử frontend (UI) ở giai đoạn này.

Bao gồm cả các tình huống hợp lệ và không hợp lệ.

6.1.3 Loại kiểm thử áp dụng

Kiểm thử chức năng (Functional Testing): Đảm bảo các API trả về đúng kết quả theo yêu cầu.

Kiểm thử hồi quy (Regression Testing): Kiểm tra lại các chức năng cũ sau khi thêm/chỉnh sửa tính năng mới.

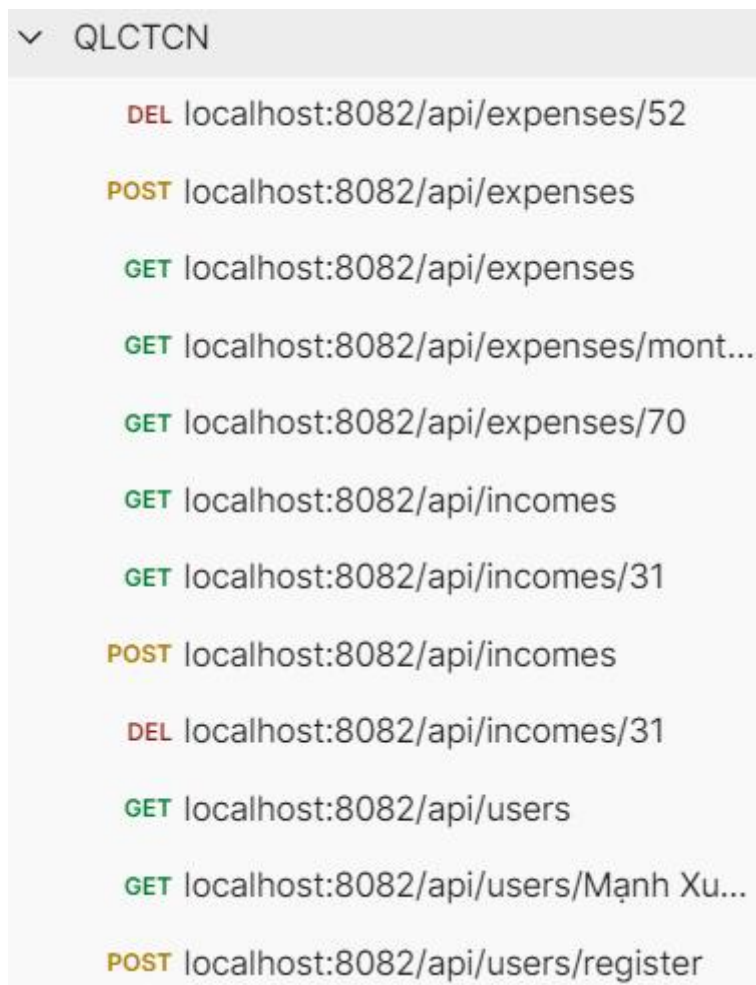
Kiểm thử tích hợp (Integration Testing): Kiểm tra luồng xử lý giữa các module (ví dụ: đăng nhập rồi mới thêm chi tiêu).

Kiểm thử tự động (Automated Testing): Tự động chạy test mỗi khi có thay đổi code bằng GitHub Actions.

6.2. Công cụ sử dụng và kết quả kiểm thử API

Tên công cụ	Mục đích
Postman	Tạo và chạy các request API thủ công, viết test case, kiểm tra phản hồi

Bảng 5 Công cụ kiểm thử API



Hình 10 Các endpoint trong dự án

6.2.1 kết quả kiểm thử API

The screenshot shows a REST client interface with a GET request to `localhost:8082/api/users`. The response is a 200 OK status with a JSON body containing two user objects. The JSON is displayed in a code editor with line numbers 1 through 11.

```
1 [
2   {
3     "id": 1,
4     "username": "khang",
5     "password": "$2a$10$XhXD1VI5114ZVc1N29p2secbt1WqRVfW8RMkwTmJjEsRVrAkg7tpS"
6   },
7   {
8     "id": 2,
9     "username": "minh thu",
10    "password": "$2a$10$aeqkKUSWJyoAdXeFiZi27u2K0.cHqxqVG1FsxynPY/ZXKqUYBBSye"
11  },
12 ]
```

Hình 11 API hiển thị tất cả user

Endpoint: `http://localhost:8082/api/users`

Phương thức: GET

Mục tiêu: Truy xuất danh sách tất cả người dùng đã đăng ký trong hệ thống

Kết quả trả về: Danh sách người dùng ở dạng JSON, mỗi người gồm:

id: mã người dùng

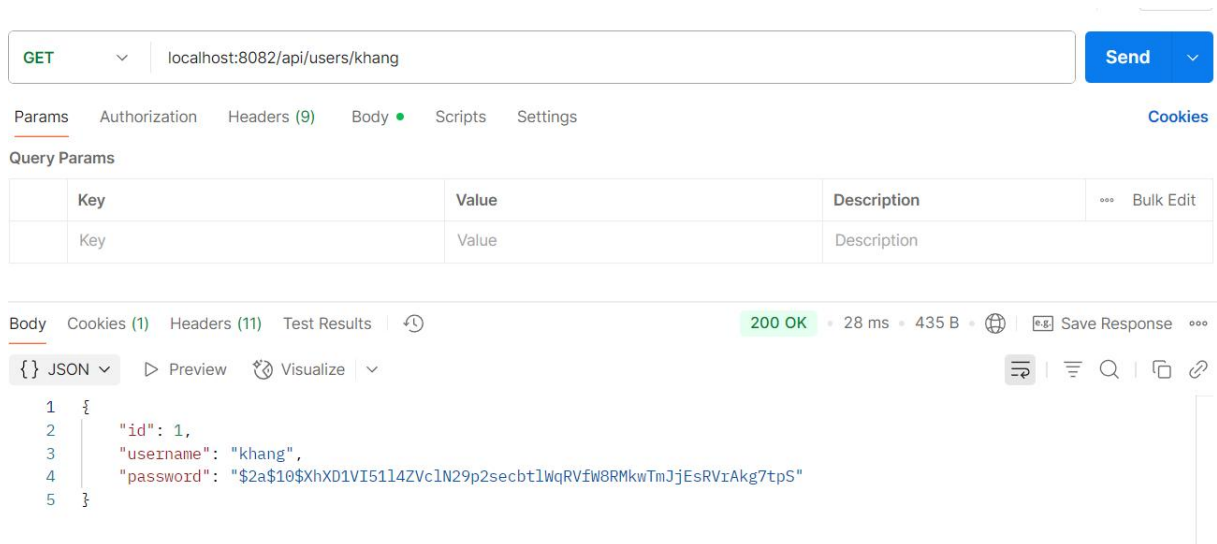
username: tên tài khoản

password: chuỗi đã mã hóa (bằng bcrypt, định dạng \$2a\$...)

API hoạt động đúng, trả về danh sách người dùng từ cơ sở dữ liệu.

Trường password được mã hóa bằng bcrypt → thể hiện bạn đã xử lý bảo mật tối thiểu khi lưu.

Không có xác thực hoặc kiểm tra quyền → đơn giản hóa để tập trung vào chức năng chính.



Hình 12 API Hiển thị user theo id

Endpoint: http://localhost:8082/api/users/khang

Phương thức: GET

Mục tiêu: Truy xuất thông tin chi tiết của người dùng có username là khang

Kết quả trả về: Một đối tượng JSON chứa:

id: mã người dùng (1)

username: "khang"

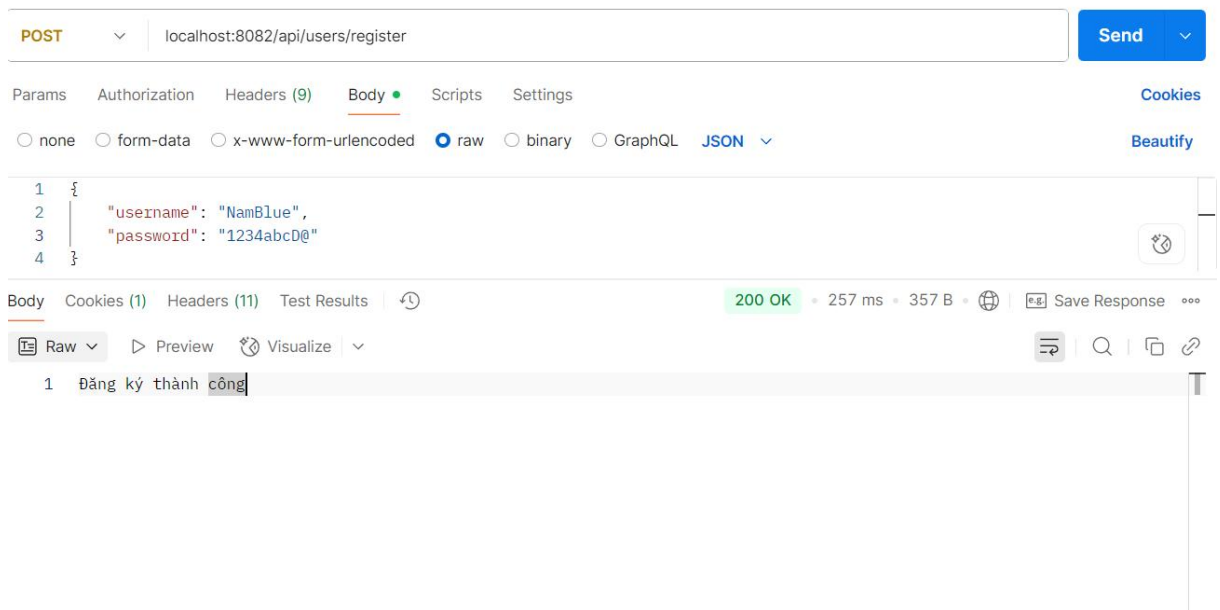
password: chuỗi mật khẩu đã được mã hóa bằng bcrypt (không phải mật khẩu gốc)

API hoạt động đúng khi trả về đúng thông tin người dùng theo tham số path username.

Trường password vẫn được mã hóa để đảm bảo không lộ mật khẩu thật.

Ứng dụng không phân quyền, nên API này không yêu cầu xác thực.

Mặc dù trả về mật khẩu mã hóa có thể không cần thiết trong ứng dụng cá nhân, nhưng vẫn đảm bảo an toàn thông tin gốc.



Hình 13 API đăng ký tài khoản

Endpoint: http://localhost:8082/api/users/register

Phương thức: POST

Dữ liệu gửi (Body JSON):

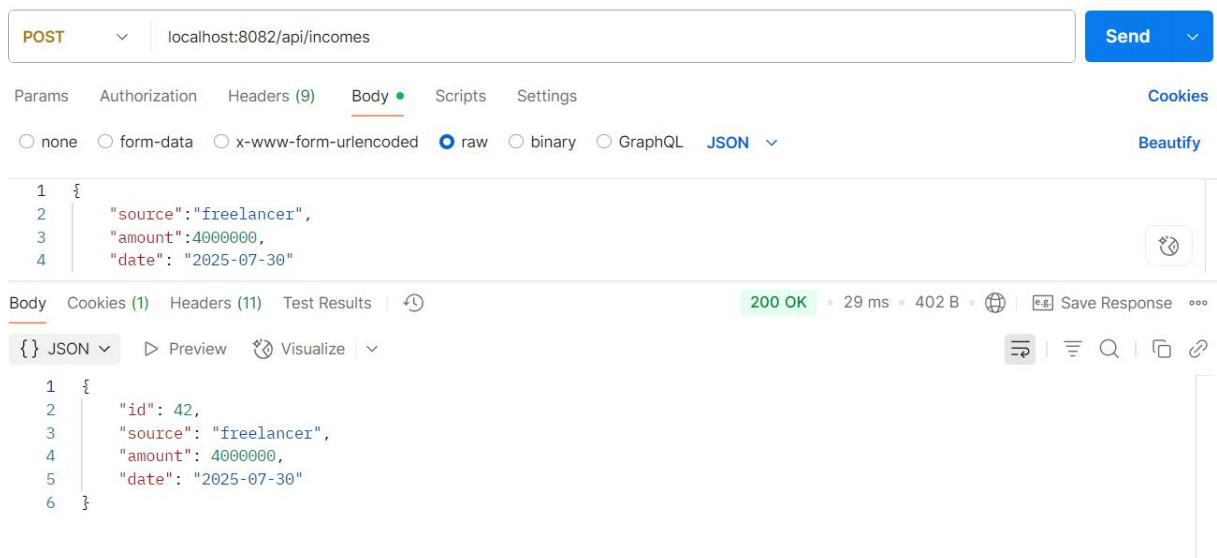
Mật khẩu người dùng được xử lý mã hóa trước khi lưu, đảm bảo bảo mật dữ liệu nhạy cảm.

API không yêu cầu xác thực để đăng ký tài khoản mới.

Ứng dụng hướng đến quản lý chi tiêu cá nhân, do đó việc đăng ký tài khoản là bước đầu tiên để sử dụng các chức năng khác.

Mô tả:

API này cho phép người dùng đăng ký tài khoản mới bằng cách gửi tên đăng nhập và mật khẩu. Mật khẩu được mã hóa trước khi lưu vào cơ sở dữ liệu để đảm bảo an toàn. API không yêu cầu xác thực, giúp người dùng dễ dàng tạo tài khoản và bắt đầu sử dụng các chức năng như ghi chép chi tiêu, thống kê tài chính và ngân sách.



Hình 14 API tạo mới thu nhập

Endpoint: http://localhost:8082/api/incomes

Phương thức: POST

Dữ liệu gửi (Body JSON):

Mục tiêu: Thêm mới một khoản thu nhập với nguồn thu, số tiền và ngày nhận

Kết quả trả về:

Đối tượng JSON của khoản thu nhập vừa tạo, bao gồm:

id: 41 (mã định danh khoản thu nhập)

source: "freelancer"

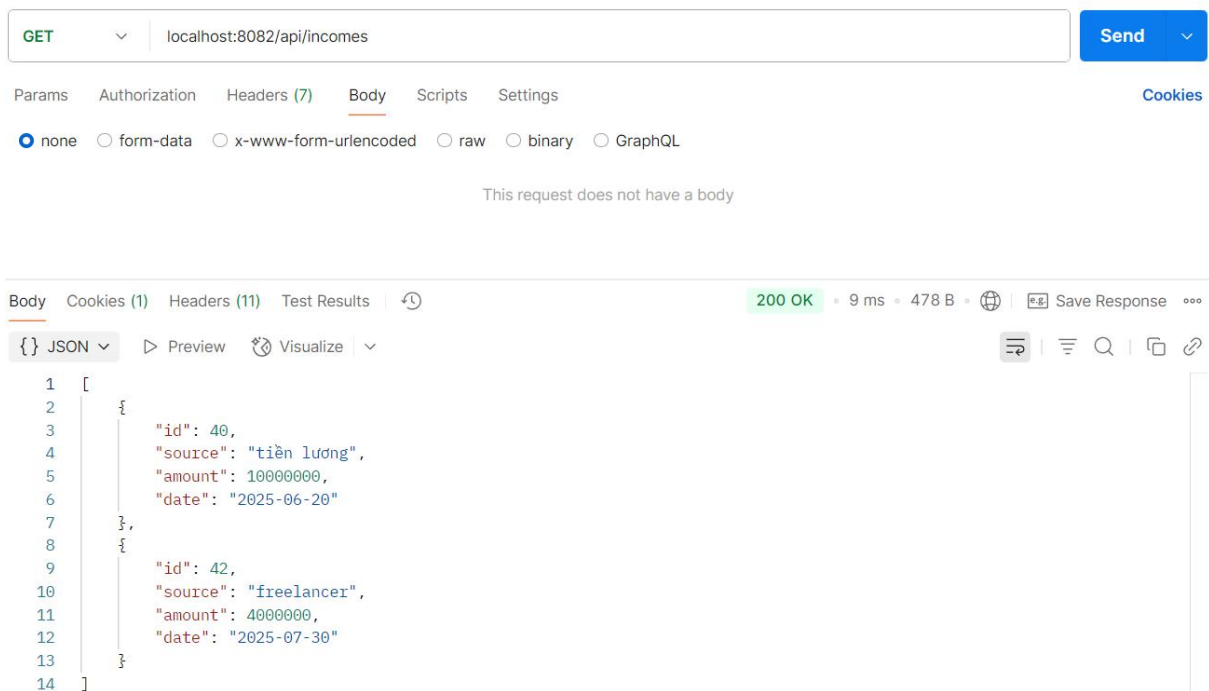
amount: 4000000

date: "2025-07-30"

API thực hiện đúng chức năng thêm dữ liệu mới vào cơ sở dữ liệu.

Dữ liệu trả về cho phép xác nhận thông tin đã được lưu chính xác.

Không yêu cầu xác thực trong ứng dụng cá nhân này.



Hình 15 API hiển thị tất cả thu nhập

Endpoint: `http://localhost:8082/api/incomes`

Phương thức: GET

Mục tiêu: Truy xuất toàn bộ các khoản thu nhập đã được thêm vào hệ thống

Kết quả trả về:

Một mảng JSON gồm các đối tượng khoản thu nhập, mỗi khoản bao gồm:

id: mã định danh khoản thu nhập

source: nguồn thu nhập (ví dụ: “tiền lương”, “freelancer”)

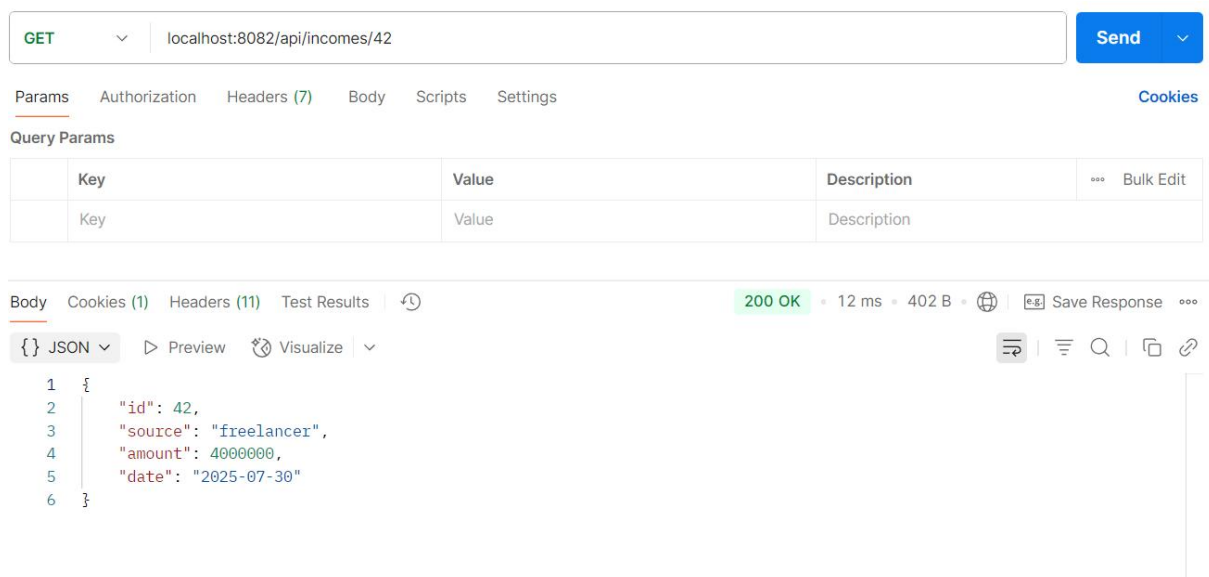
amount: số tiền thu nhập

date: ngày nhận thu nhập

API hoạt động ổn định, trả về đầy đủ danh sách khoản thu nhập cho người dùng.

Dữ liệu trả về theo đúng định dạng JSON tiêu chuẩn, dễ dàng xử lý cho phần front-end.

Ứng dụng không yêu cầu xác thực trong giai đoạn hiện tại.



Hình 16 API hiển thị thu nhập theo id

Endpoint: http://localhost:8082/api/expenses/42

Phương thức: GET

Mục tiêu: Truy vấn thông tin chi tiết của một khoản chi tiêu theo id. với id = 78, hệ thống sẽ trả về thông tin tương ứng với khoản chi có mã số 78.

Trường hợp không tìm thấy id, hệ thống có thể trả về mã lỗi 404 Not Found.

Endpoint này thường được sử dụng trong chức năng "xem chi tiết" trên giao diện người dùng.

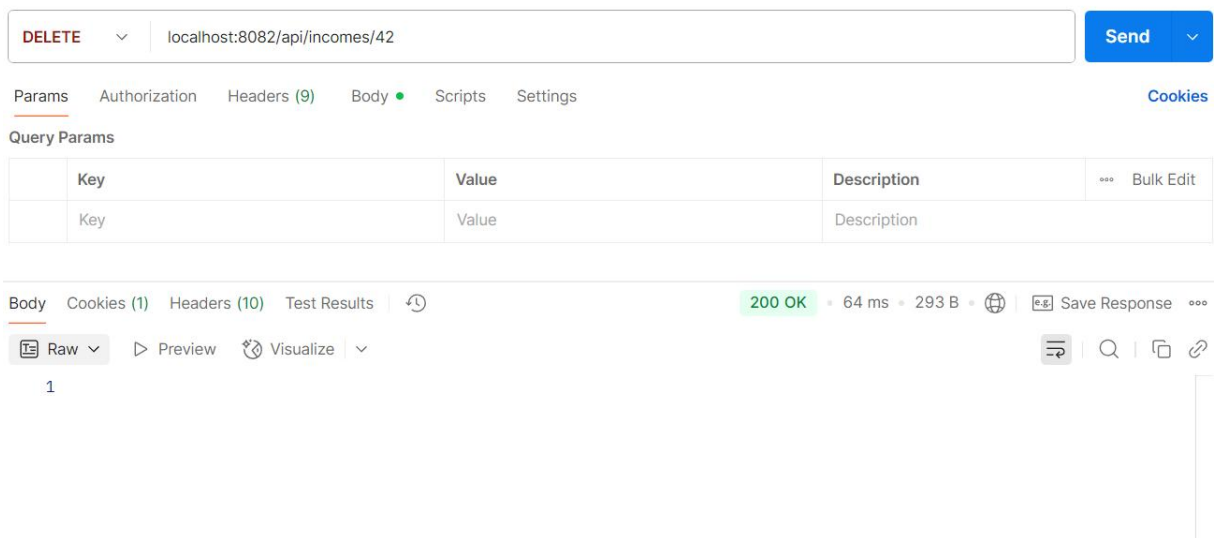
Mô tả chi tiết:

API này được sử dụng để lấy thông tin cụ thể của một khoản thu nhập dựa trên mã số (ID).

Trong ví dụ, ID là 42, và hệ thống trả về thông tin thu nhập từ công việc freelancer với số tiền là 4.000.000 VNĐ vào ngày 30/07/2025.

Nếu ID không tồn tại, hệ thống sẽ trả về mã lỗi 404 Not Found.

Endpoint này thường được tích hợp trong chức năng “xem chi tiết” trên giao diện người dùng, giúp người dùng kiểm tra lại các khoản thu nhập đã ghi nhận.



Hình 17 API xóa thu nhập

Endpoint: http://localhost:8082/api/incomes/42

Phương thức: DELETE

Mục tiêu: Xóa khoản thu nhập có id = 41 khỏi hệ thống

Kết quả trả về:

HTTP status: 200 OK (hoặc 204 No Content nếu không trả về dữ liệu)

Xác nhận rằng khoản thu nhập đã bị xóa thành công khỏi cơ sở dữ liệu

API thực hiện đúng chức năng xóa dữ liệu theo ID.

Sau khi xóa, nếu thực hiện lại GET /api/incomes, khoản có id = 41 sẽ không còn xuất hiện.

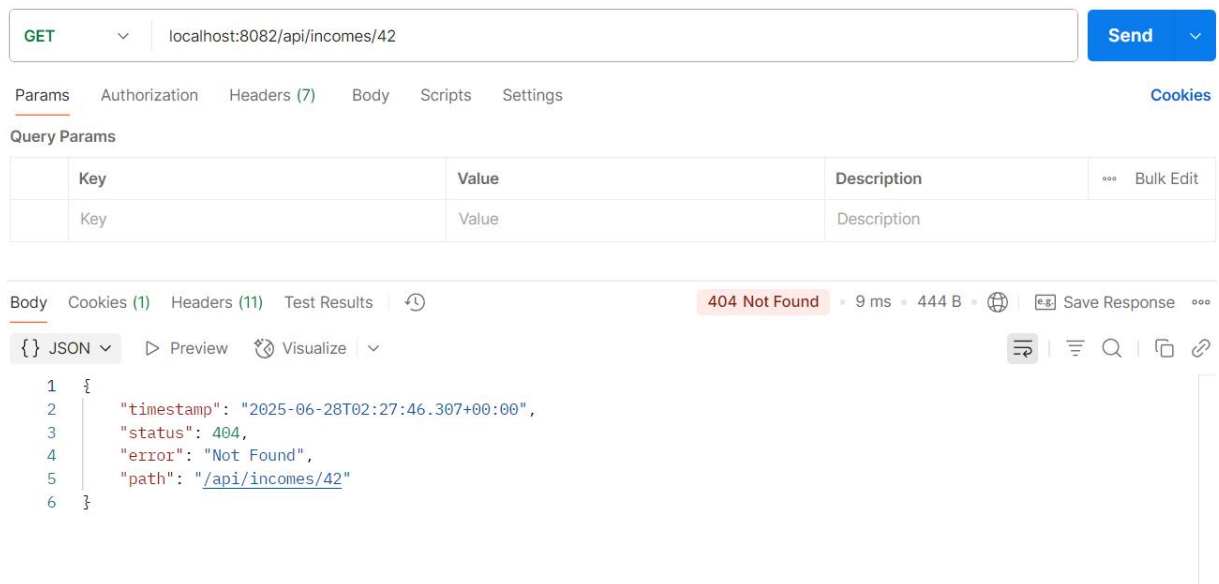
Không có xác thực hoặc phân quyền — phù hợp với ứng dụng cá nhân đơn giản.

Nếu ID không tồn tại, hệ thống nên trả về HTTP 404

Mô tả chi tiết:

API này thực hiện chức năng xóa dữ liệu theo ID, phù hợp với các thao tác quản lý thu nhập cá nhân.

Không yêu cầu phân quyền, phù hợp với ứng dụng đơn giản dành cho cá nhân. Nếu ID không tồn tại, hệ thống nên trả về mã lỗi 404 Not Found.



Hình 18 API thu nhập không tìm thấy

Endpoint: http://localhost:8082/api/incomes/42

Phương thức: GET

Mô tả chi tiết:

Đây là phản hồi đúng kỳ vọng sau khi khoản thu nhập có id = 42 đã bị xóa khỏi hệ thống.

Hệ thống trả về mã lỗi 404 Not Found, cho biết không tìm thấy tài nguyên tương ứng.

Thông báo lỗi rõ ràng, bao gồm:

timestamp: thời điểm xảy ra lỗi

status: mã lỗi HTTP

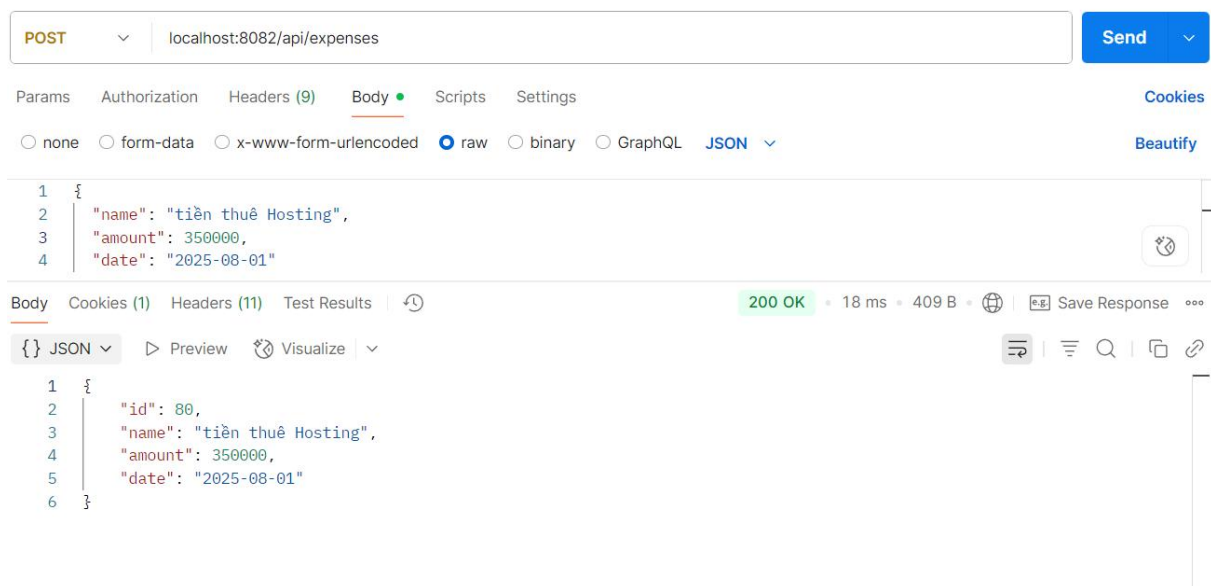
error: loại lỗi

path: đường dẫn gây lỗi

Đây là kết quả đúng kỳ vọng: sau khi khoản có id = 41 đã bị xóa, hệ thống không còn tìm thấy nên trả về lỗi 404.

Thông báo lỗi rõ ràng, có timestamp, mã lỗi (status = 404), loại lỗi (Not Found) và đường dẫn gây lỗi (/api/incomes/41).

Việc xử lý lỗi đúng giúp tăng độ tin cậy và dễ debug cho ứng dụng.



Hình 19 API tạo mới chi tiêu

Endpoint: <http://localhost:8082/api/expenses>

Phương thức: POST

Dữ liệu gửi (Body JSON):

Mục tiêu: Thêm một khoản chi tiêu với nội dung, số tiền và ngày phát sinh

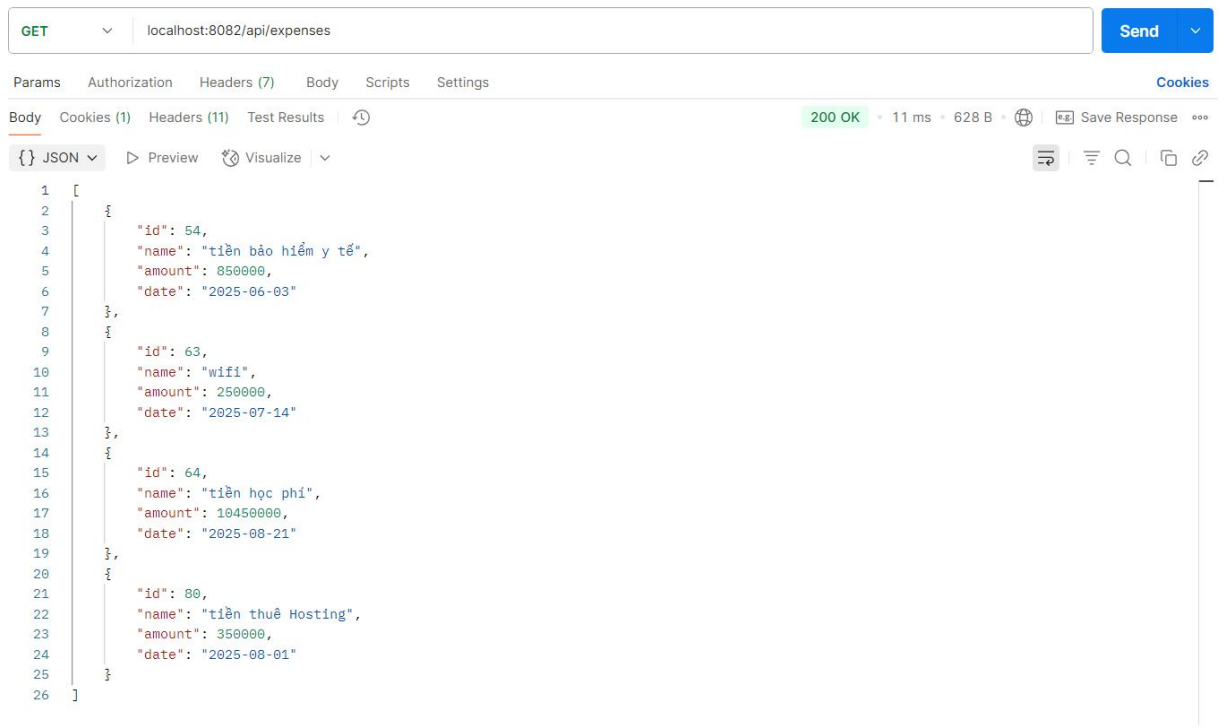
Mô tả chi tiết:

API hoạt động đúng, trả về thông tin khoản chi tiêu vừa được thêm vào, bao gồm mã số id được hệ thống tự động tạo.

Các trường thông tin phản hồi khớp hoàn toàn với dữ liệu đã gửi.

Trường ngày (date) được định dạng theo chuẩn ISO (YYYY-MM-DD), đảm bảo tính nhất quán và dễ xử lý.

Ứng dụng hiện không yêu cầu xác thực, phù hợp với mục tiêu sử dụng cá nhân và đơn giản hóa quy trình kiểm thử.



Hình 20 API hiển thị tất cả chi tiêu

Endpoint: http://localhost:8082/api/expenses

Phương thức: GET

Mục tiêu: Truy xuất toàn bộ các khoản chi tiêu đã được thêm vào hệ thống

Kết quả trả về: Một mảng JSON các khoản chi tiêu, mỗi khoản bao gồm:

id: mã định danh khoản chi

name: tên khoản chi (ví dụ: “tiền bảo hiểm y tế”, “wifi”)

amount: số tiền đã chi

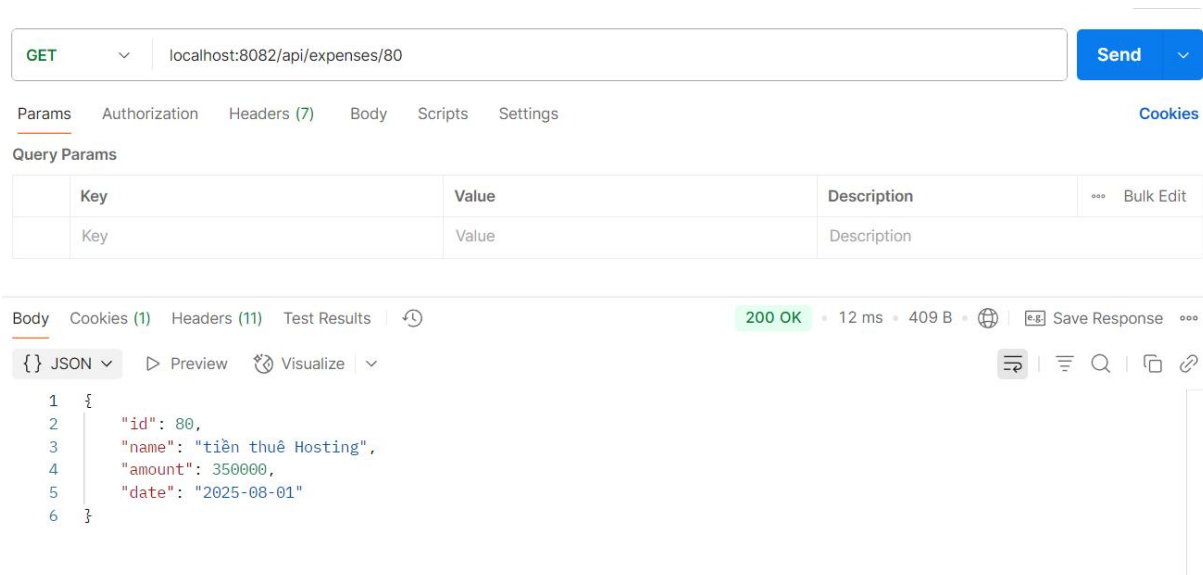
date: ngày chi (định dạng YYYY-MM-DD)

API phản hồi đúng dữ liệu đã thêm, thể hiện hệ thống lưu trữ và truy vấn chính xác.

Các khoản chi tiêu hiển thị đầy đủ thông tin, sắp xếp theo thứ tự id hoặc theo thứ tự thêm (tùy vào backend).

Dữ liệu phản hồi định dạng JSON chuẩn, thuận tiện xử lý ở phía frontend.

Không yêu cầu xác thực – phù hợp với ứng dụng đơn giản dành cho cá nhân.



Hình 21 API hiển thị chi tiêu theo id

Endpoint: <http://localhost:8082/api/expenses/78>

Phương thức: GET

Mục tiêu: Truy vấn thông tin chi tiết của một khoản chi tiêu theo id.
với id = 78, hệ thống sẽ trả về thông tin tương ứng với khoản chi có mã số 78.

Trường hợp không tìm thấy id, hệ thống có thể trả về mã lỗi 404 Not Found.

Endpoint này thường được sử dụng trong chức năng "xem chi tiết" trên giao diện người dùng.

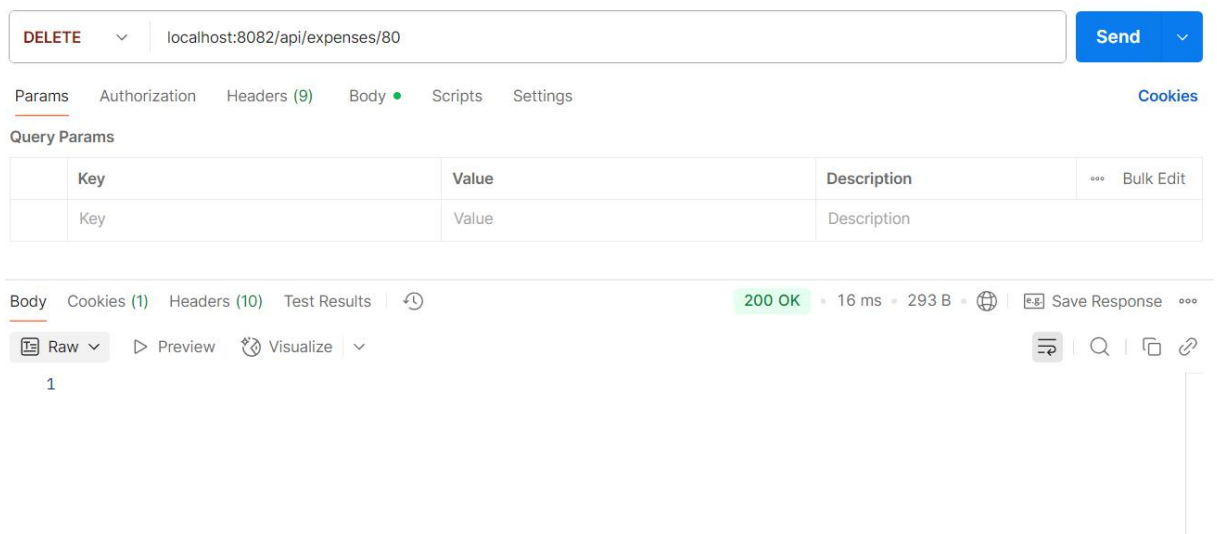
Mô tả chi tiết:

API này cho phép truy xuất thông tin chi tiết của một khoản chi tiêu cụ thể dựa trên ID.

Trong ví dụ, ID là 80, và hệ thống trả về thông tin chi tiêu cho dịch vụ thuê Hosting với số tiền là 350.000 VNĐ vào ngày 01/08/2025.

Nếu ID không tồn tại, hệ thống sẽ trả về mã lỗi 404 Not Found.

Endpoint này thường được sử dụng trong chức năng “xem chi tiết” trên giao diện người dùng, giúp người dùng kiểm tra lại các khoản chi đã ghi nhận.



Hình 22 API xóa chi tiêu

Endpoint: `/api/expenses/{id}`

Phương thức: DELETE

Kiểu dữ liệu gửi đi:

Không có body.

Tham số duy nhất là id được truyền trực tiếp trên URL để xác định khoản chi cần xóa.

Mục tiêu:

Xóa một khoản chi tiêu cụ thể theo id.

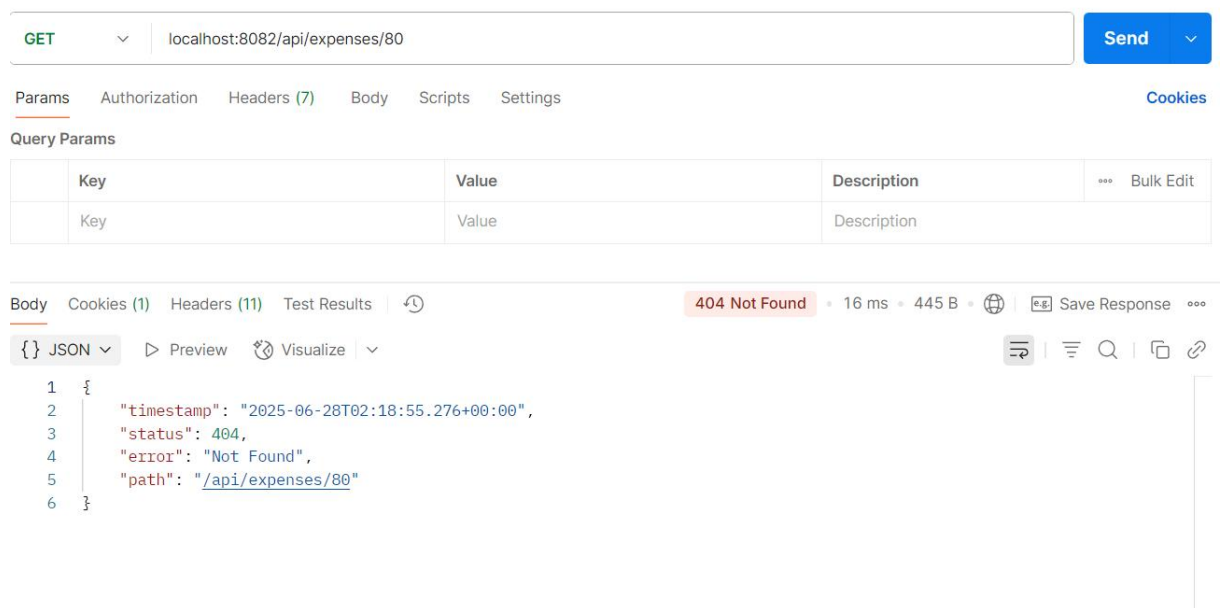
Ví dụ: Xóa khoản chi có mã số 80 khỏi cơ sở dữ liệu.

Khi người dùng xác nhận xóa trên giao diện, hệ thống sẽ gửi yêu cầu DELETE đến endpoint này.

Nếu không tìm thấy khoản chi với id đã cho, hệ thống có thể trả về mã lỗi 404 Not Found.

Mô tả chi tiết:

Khi người dùng xác nhận xóa, hệ thống gửi yêu cầu DELETE. Nếu khoản chi tồn tại, sẽ xóa và trả về 200 OK hoặc 204 No Content; nếu không, trả về 404 Not Found. API không yêu cầu xác thực, phù hợp cho ứng dụng cá nhân hoặc kiểm thử.



Hình 23 API chi tiêu không tìm thấy

Mục tiêu: Thông báo lỗi khi hệ thống không tìm thấy khoản chi tiêu tương ứng với ID được yêu cầu.

Khi người dùng gửi yêu cầu GET, DELETE, hoặc PUT đến endpoint `/api/expenses/{id}` với một ID không tồn tại trong cơ sở dữ liệu.

Mô tả chi tiết:

Hệ thống trả về mã lỗi 404 Not Found, cho biết không tìm thấy tài nguyên tương ứng.

Thông báo lỗi rõ ràng, bao gồm:

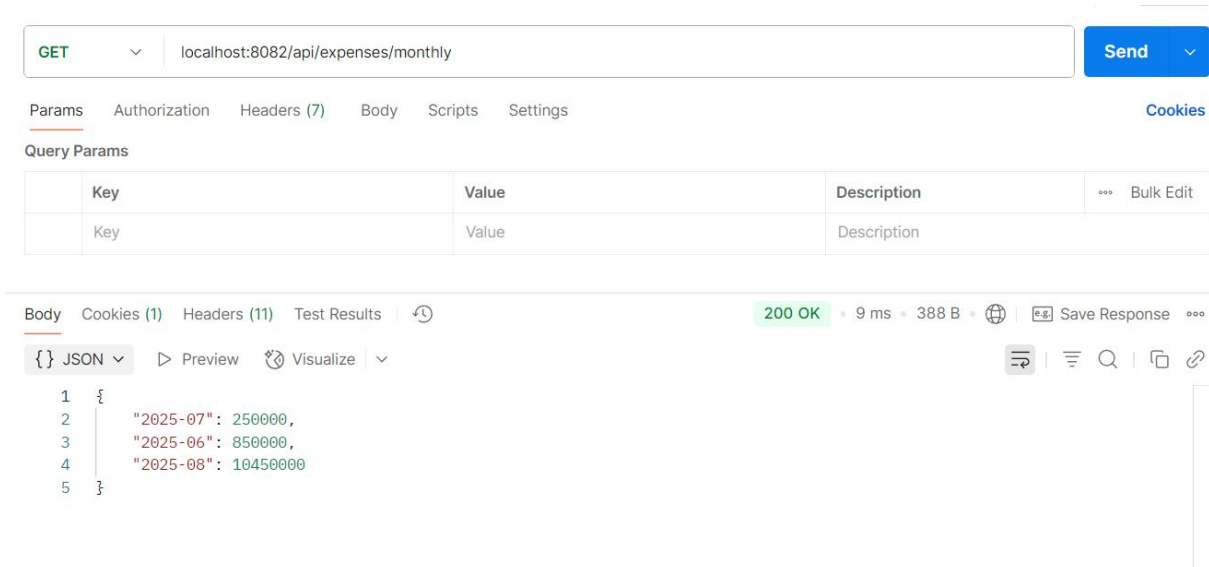
timestamp: thời điểm xảy ra lỗi

status: mã lỗi HTTP

error: loại lỗi

path: đường dẫn gây lỗi

Việc xử lý lỗi đúng cách giúp cải thiện trải nghiệm người dùng và hỗ trợ quá trình kiểm thử, gỡ lỗi hiệu quả hơn. Lỗi này để thông báo cho người dùng rằng khoản chi yêu cầu không tồn tại. Có thể xảy ra với các yêu cầu GET, DELETE, PUT khi tham số id không hợp lệ hoặc không tìm thấy.



Hình 24 API tổng chi tiêu từng tháng

Endpoint:/api/expenses/monthly

Phương thức:GET

Mục tiêu: Truy xuất tổng số tiền chi tiêu của từng tháng.

API trả về dữ liệu dạng JSON, trong đó mỗi khóa là tháng (YYYY-MM), giá trị là tổng số tiền chi trong tháng đó.

API này phục vụ cho chức năng thống kê chi tiêu theo tháng, giúp người dùng có cái nhìn tổng quan về mức chi tiêu của mình.

Có thể dùng dữ liệu này để vẽ biểu đồ hoặc báo cáo tài chính hàng tháng.

Nếu không có dữ liệu chi trong tháng nào đó, tháng đó sẽ không xuất hiện trong kết quả trả về.

Mô tả chi tiết:

API trả về dữ liệu JSON với key là tháng (YYYY-MM) và value là tổng chi tiêu trong tháng (VNĐ). Dùng cho thống kê chi tiêu theo tháng, hỗ trợ vẽ biểu đồ và lập báo cáo. Tháng không có dữ liệu sẽ không xuất hiện trong kết quả.

CHƯƠNG 7. ĐÁNH GIÁ VÀ KẾT LUẬN

7.1. Những khó khăn gặp phải trong quá trình thực hiện

Hạn chế của kiến trúc monolithic trong mở rộng và bảo trì

Kiến trúc monolithic có ưu điểm là đơn giản về mặt triển khai và phát triển ban đầu, tuy nhiên khi ứng dụng phát triển lớn hơn, việc bảo trì và mở rộng có thể gặp khó khăn do tất cả các thành phần gắn kết chặt chẽ trong một khối duy nhất. Điều này làm giảm tính linh hoạt trong việc cập nhật, mở rộng hoặc sửa lỗi mà không ảnh hưởng đến toàn bộ hệ thống.

Thiết kế cơ sở dữ liệu phù hợp với yêu cầu quản lý chi tiêu cá nhân

Mặc dù ứng dụng dành cho cá nhân với dữ liệu không quá lớn, việc thiết kế bảng, quan hệ giữa các bảng để tối ưu truy vấn và dễ dàng mở rộng là một thách thức, đặc biệt khi cần đảm bảo tính toàn vẹn dữ liệu.

Bảo mật thông tin cá nhân trong ứng dụng đơn người dùng

Việc bảo vệ dữ liệu cá nhân, bao gồm thông tin đăng nhập và dữ liệu chi tiêu tài chính, đòi hỏi áp dụng các biện pháp bảo mật như mã hóa mật khẩu, quản lý phiên đăng nhập an toàn, tránh rò rỉ dữ liệu dù ứng dụng không có nhiều người dùng.

Thiết kế giao diện đơn giản, thân thiện và dễ sử dụng

Giao diện cần được xây dựng sao cho người dùng cá nhân có thể dễ dàng nhập liệu, theo dõi và phân tích chi tiêu mà không gặp khó khăn trong thao tác. Việc cân bằng giữa đầy đủ tính năng và sự đơn giản là thử thách trong thiết kế UX/UI

Quản lý và kiểm soát phiên bản mã nguồn trong dự án cá nhân

Mặc dù dự án có quy mô nhỏ, việc quản lý mã nguồn khoa học vẫn rất cần thiết để tránh lỗi phát sinh do thay đổi code và giúp việc bảo trì, nâng cấp trở nên dễ dàng hơn.

7.2. Bài học rút ra và đề xuất cải thiện trong tương lai

Lựa chọn kiến trúc phù hợp theo quy mô dự án

Khi bắt đầu xây dựng một ứng dụng, việc lựa chọn kiến trúc hệ thống phù hợp với quy mô và định hướng phát triển lâu dài là điều quan trọng. Đối với các dự án nhỏ, ứng dụng cá nhân hoặc trong giai đoạn thử nghiệm ý tưởng, kiến trúc monolithic (nguyên khối) thường là lựa chọn ưu tiên vì tính đơn giản trong thiết kế, triển khai và vận hành. Tất cả các thành phần được gói gọn trong một khối thống nhất giúp quá trình phát triển ban đầu nhanh chóng và tiết kiệm chi phí.

Tuy nhiên, khi ứng dụng bắt đầu có xu hướng mở rộng, gia tăng tính năng hoặc phục vụ nhiều người dùng hơn, kiến trúc monolithic dễ trở nên cồng kềnh, khó kiểm soát. Lúc này, có thể xem xét chuyển sang kiến trúc microservices (dịch vụ nhỏ độc lập) hoặc modular (module hóa). Những kiến trúc này cho phép chia nhỏ hệ thống thành các phần riêng biệt, dễ phát triển độc lập, dễ bảo trì và mở rộng theo nhu cầu. Việc xác định trước định hướng quy mô sẽ giúp lựa chọn kiến trúc phù hợp ngay từ đầu, tiết kiệm thời gian và công sức về sau.

Tối ưu hóa thiết kế cơ sở dữ liệu và truy vấn

Thiết kế cơ sở dữ liệu cần đảm bảo nguyên tắc chuẩn hóa, giảm thiểu dữ liệu dư thừa và tối ưu truy vấn nhằm nâng cao hiệu suất, đặc biệt khi dữ liệu phát triển lâu dài.

Đảm bảo bảo mật ngay từ đầu

Áp dụng các biện pháp bảo mật cơ bản và chuẩn mực như mã hóa mật khẩu, sử dụng HTTPS, quản lý phiên đăng nhập an toàn, để bảo vệ dữ liệu cá nhân và tăng độ tin cậy của ứng dụng.

Phát triển giao diện người dùng dựa trên trải nghiệm thực tế

Tập trung xây dựng giao diện đơn giản, trực quan, hỗ trợ người dùng dễ dàng nhập liệu và quản lý thông tin. Có thể áp dụng các phương pháp test UX để cải thiện trải nghiệm người dùng.

Quản lý mã nguồn và quy trình phát triển chuyên nghiệp

Ngay cả khi làm việc cá nhân, việc áp dụng các công cụ quản lý mã nguồn như Git sẽ giúp theo dõi lịch sử thay đổi, tránh mất mát dữ liệu và dễ dàng khôi phục khi gặp lỗi. Bên cạnh đó, chia nhánh (branching) và đặt tên commit rõ ràng là thói quen tốt giúp mã nguồn dễ hiểu và cộng tác hiệu quả nếu sau này mở rộng nhóm phát triển.

Song song với quản lý mã nguồn, nên xây dựng quy trình kiểm thử đơn vị (unit test), giúp kiểm tra từng thành phần trong ứng dụng một cách độc lập. Việc này giúp phát hiện lỗi sớm, giảm chi phí sửa chữa và đảm bảo chất lượng phần mềm. Dù quy mô nhỏ, nhưng cách tiếp cận chuyên nghiệp từ đầu sẽ tạo nền tảng vững chắc, giảm thiểu rủi ro và thuận lợi hơn nếu mở rộng, nâng cấp trong tương lai.

CHƯƠNG 8. PHỤ LỤC

8.1. Hướng dẫn cài đặt và chạy ứng dụng

8.1.1. Cài đặt môi trường

Cài đặt JDK

Truy cập: trang chính thức của Oracle:

<https://www.oracle.com/java/technologies/downloads/#java21>

-Chọn hệ điều hành tương ứng (Windows/macOS/Linux).

-Tải xuống và cài đặt JDK 21.

8.1.2. Cài đặt XAMPP

Truy cập trang: <https://www.apachefriends.org/download.html>

Tải phiên bản phù hợp với hệ điều hành và cài đặt

Mở **XAMPP Control Panel**, bật:

Apache

MySQL

8.1.3. Cài đặt cơ sở dữ liệu

Truy cập: <http://localhost/phpmyadmin>

Chọn **Database** → Nhập tên: quan_ly_chi_tieu_ca_nhan → Bấm **Create**.

Chọn **Import** → Tải lên file quan_ly_chi_tieu_ca_nhan.sql từ thư mục dự án
Bấm **Go** để import.

8.1.4. Cài đặt Postman

Truy cập trang chính thức: <https://www.postman.com/downloads/>

Tải bản dành cho hệ điều hành đang sử dụng (Windows/macOS/Linux)

Cài đặt theo hướng dẫn và đăng nhập (có thể dùng Google để đăng nhập nhanh)

8.1.5. Clone Ứng Dụng & Truy Cập

```
git clone https://github.com/haivoDA22TTD/CNPM_QLCTCN_Spring-Boot  
cd CNPM_QLCTCN_Spring-Boot
```

8.1.6. Cài Đặt Thư Viện Phụ Thuộc

Chạy lệnh sau để cài đặt toàn bộ các thư viện cần thiết của dự án:

```
mvn clean install
```

8.1.7. Khởi Động Ứng Dụng

Sử dụng Maven để chạy ứng dụng Spring Boot:

```
mvn spring-boot:run
```

8.1.8. Ứng dụng sẽ chạy tại địa chỉ:

<http://localhost:8082>

8.2. Link GitHub và repository

https://github.com/haivoDA22TTD/CNPM_QLCTCN_Spring-Boot